

# Corrigé — Examen d'entraînement LEPL1110 — Éléments Finis

Claude code

Mai 2026

## Exercice 1 — Équation de diffusion de la chaleur en 1D

### 1.1 — Formulation faible

On multiplie  $\partial_t u - \kappa \partial_{xx} u = 0$  par  $v \in H^1(0, 1)$  et on intègre :

$$\int_0^1 \partial_t u v \, dx - \kappa \int_0^1 \partial_{xx} u v \, dx = 0.$$

IBP :  $-\kappa \int_0^1 u'' v \, dx = -\kappa [u'v]_0^1 + \kappa \int_0^1 u' v' \, dx$ . Le terme de bord vaut  $-\kappa(u'(1, t)v(1) - u'(0, t)v(0)) = 0$  car  $u'(0, t) = u'(1, t) = 0$  (conditions de Neumann). On obtient :

$$\int_0^1 \partial_t u v \, dx + \kappa \int_0^1 \partial_x u \partial_x v \, dx = 0, \quad \forall v \in H^1(0, 1).$$

Les conditions de Neumann sont dites **naturelles** : elles disparaissent dans le terme de bord et n'imposent aucune contrainte sur  $v$  ni sur l'espace fonctionnel.

**Erreur classique :** Croire que les termes de bord s'annulent parce que  $v = 0$  en bord. Ici c'est  $u' = 0$  (conditions de Neumann) qui les annule. Les conditions de Dirichlet s'imposeraient en restreignant l'espace de  $v$ .

### 1.2 — Semi-discrétisation

En substituant  $u_h = \sum_j u_j(t) \varphi_j$  et  $v = \varphi_i$  :

$$\sum_j \dot{u}_j \underbrace{\int_0^1 \varphi_j \varphi_i \, dx}_{M_{ij}} + \kappa \sum_j u_j \underbrace{\int_0^1 \varphi_j' \varphi_i' \, dx}_{K_{ij}} = 0, \quad \forall i.$$

En notation matricielle :  $M \dot{\mathbf{u}}(t) + \kappa K \mathbf{u}(t) = \mathbf{0}$ .

### 1.3 — Matrices élémentaires et assemblage ( $N = 2$ )

(a) Sur un élément  $[x_e, x_{e+1}]$  de longueur  $h$ , avec  $\varphi_0 = (x_{e+1} - x)/h$ ,  $\varphi_1 = (x - x_e)/h$  :

$$K^{(e)} = \frac{1}{h} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad M^{(e)} = \frac{h}{6} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

(b) Assemblage global avec  $h = 1/2$  :

$$K = \begin{pmatrix} 2 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 2 \end{pmatrix}, \quad M = \frac{1}{12} \begin{pmatrix} 2 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 2 \end{pmatrix}.$$

(c)  $K\mathbf{1} = (2 - 2, -2 + 4 - 2, -2 + 2)^\top = (0, 0, 0)^\top$ . ✓

Interprétation : un champ de température uniforme ( $u = \text{cst}$ ) ne génère aucun flux de chaleur.

## 1.4 — Schéma $\theta$

(a) Le schéma  $\theta$  s'écrit :

$$(M + \theta \Delta t \kappa K) \mathbf{u}^{n+1} = (M - (1 - \theta) \Delta t \kappa K) \mathbf{u}^n.$$

$\theta = 0$  : Euler explicite ;  $\theta = 1$  : Euler implicite ;  $\theta = 1/2$  : Crank–Nicolson.

(b)  $\frac{d}{dt} \|u\|_{L^2}^2 = 2 \int_0^1 \partial_t u u \, dx = -2\kappa \int_0^1 (u')^2 \, dx \leq 0$ .

Le schéma  $\theta$  est inconditionnellement  $L^2$ -stable pour  $\theta \geq 1/2$ .

(c) Pour  $\theta = 0$ , la condition de stabilité est  $\Delta t \leq \frac{2}{\kappa \lambda_{\max}}$ .

## 1.5 — Analyse modale

(a)  $\phi_n(x) = \cos(n\pi x)$ ,  $\lambda_n = (n\pi)^2$ ,  $n = 0, 1, 2, \dots$

(b) Pour  $u_0(x) = \cos(\pi x)$  :  $u(x, t) = \cos(\pi x) e^{-\kappa\pi^2 t}$ .

(c) Temps caractéristique  $\tau = 1/(\kappa\pi^2)$ . La température se stabilise vers la moyenne  $\bar{u}_0 = \int_0^1 u_0 \, dx = 0$ .

**Erreur classique** : Oublier le mode  $n = 0$  (température uniforme). La solution tend vers 0 ici parce que  $\int_0^1 \cos(\pi x) \, dx = 0$ , pas parce que toute solution tend vers 0.

## Exercice 2 — Corrigé : le jeu des erreurs

### ① — Mauvais nombre de nœuds

```
1 x = np.linspace(0.0, 1.0, N)           # ERREUR : N noeuds (manque le
   dernier)
2 x = np.linspace(0.0, 1.0, N + 1)       # CORRECT : N+1 noeuds pour N
   elements P1
```

$N$  éléments  $P_1$  nécessitent  $N + 1$  nœuds.

### ② — Jacobien incorrect

```
1 Jac = h           # ERREUR : jacobien de [-1,1] -> [x_e, x_e+h] vaut h/2
2 Jac = h / 2.0     # CORRECT
```

### ③ et ④ — Fonctions de forme échangées

```
1 Nq[0] = 0.5*(1.0 + xi)    # ERREUR : Nq[0] associe au noeud GAUCHE (
    xi=-1)
2 Nq[1] = 0.5*(1.0 - xi)    # ERREUR : Nq[1] associe au noeud DROIT (
    xi=+1)
3 # CORRECT :
4 Nq[0] = 0.5*(1.0 - xi)    # N0 = 1 en xi=-1
5 Nq[1] = 0.5*(1.0 + xi)    # N1 = 1 en xi=+1
```

### ⑤ — Division au lieu de multiplication

```
1 dN_dx[i] = dN_dxi[i] / dxi_dx    # ERREUR : doit multiplier
2 dN_dx[i] = dN_dxi[i] * dxi_dx    # CORRECT : dN/dx = dN/dxi * dxi/dx
```

### ⑥ — Indice $J$ figé

```
1 J = dof[i]    # ERREUR : J doit varier avec j
2 J = dof[j]    # CORRECT
```

### ⑦ — Ordre de retour inversé

```
1 return K, M    # ERREUR : la spec demande (M, K)
2 return M, K    # CORRECT
```

## Code corrigé

```
1 import numpy as np
2
3 def assemble_heat_1d(N, kappa):
4     x = np.linspace(0.0, 1.0, N + 1)    # (1) corrige
5
6     M = np.zeros((N + 1, N + 1))
7     K = np.zeros((N + 1, N + 1))
8
9     xi_q = [-1.0/np.sqrt(3.0), 1.0/np.sqrt(3.0)]
10    w_q = [1.0, 1.0]
11    dN_dxi = [-0.5, 0.5]
12
13    for e in range(N):
14        x0 = x[e]; x1 = x[e+1]; h = x1 - x0
15        Jac = h / 2.0    # (2) corrige
16        dxi_dx = 1.0 / Jac
17        dof = [e, e+1]
18
19        for q in range(2):
20            xi = xi_q[q]; w = w_q[q]
```

```

21     Nq = [0.5*(1.0 - xi), 0.5*(1.0 + xi)] # (3)(4)
22     corrige
23     dN_dx = [dN_dxi[i] * dxi_dx for i in range(2)] # (5)
24     corrige
25     for i in range(2):
26         I = dof[i]
27         for j in range(2):
28             J = dof[j] # (6) corrige
29             M[I,J] += Nq[i]*Nq[j]*w*Jac
30             K[I,J] += kappa*dN_dx[i]*dN_dx[j]*w*Jac
31     return M, K # (7) corrige

```